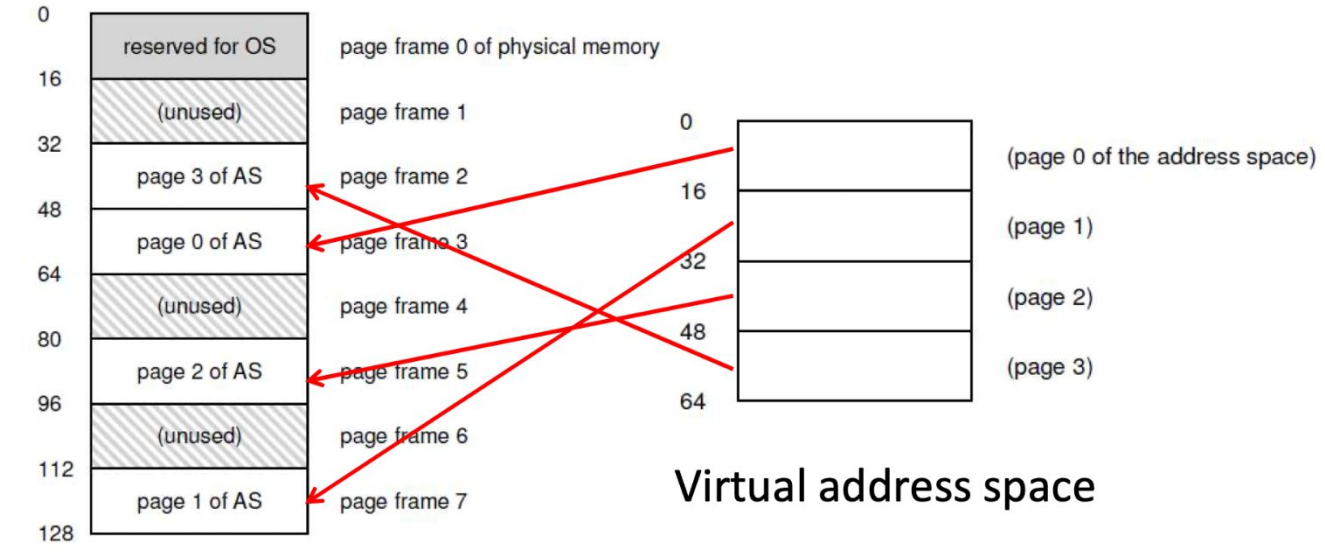
Paging

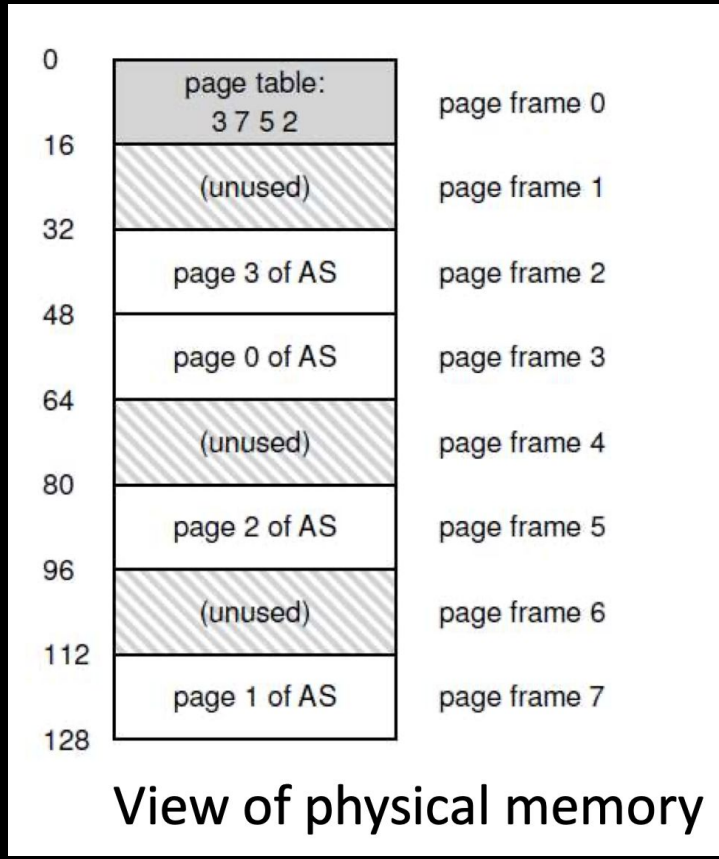


Page to frame mappings:

0 → 3
1 → 7
2 → 5
3 → 2

Page table

- Per process data structure to translate virtual address (VA) to physical address (PA)
- Stores frame numbers for all pages of a process in array
- [3 7 5 2] corresponding to pages 0 to 3 of the process
  - *Part of OS memory (in PCB)*
- MMU has access to page table of current process, uses it for address translation



Address translation using paging

- Address translation performed by MMU using page table
- Most significant bits (MSB) of VA give virtual page number, least significant bits (LSB) give offset within page
- Page table maps virtual page number (VPN) to physical frame number (PFN)
- MMU maps VPN to PFN, adds offset to get PA
- Location of page table of currently running process known to MMU
- Written into special CPU register by the OS, updated on every context switch/page table change

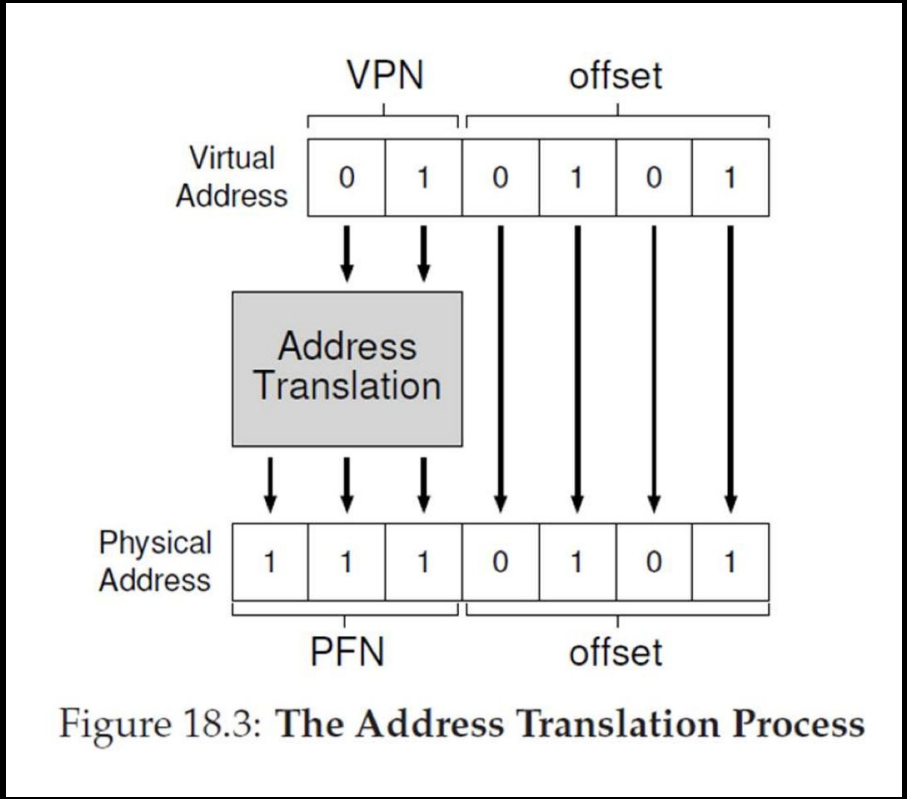
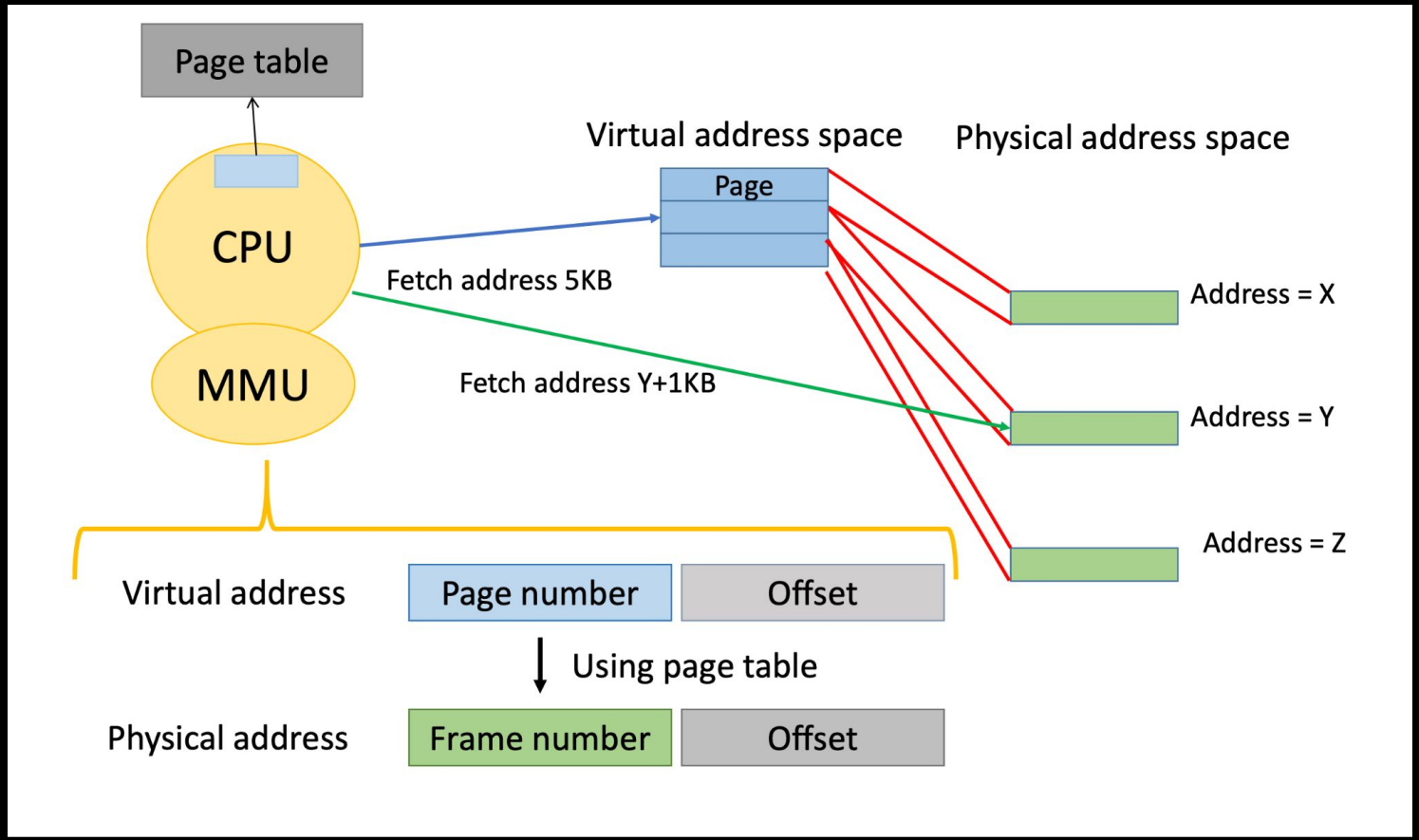


Figure 18.3: **The Address Translation Process**

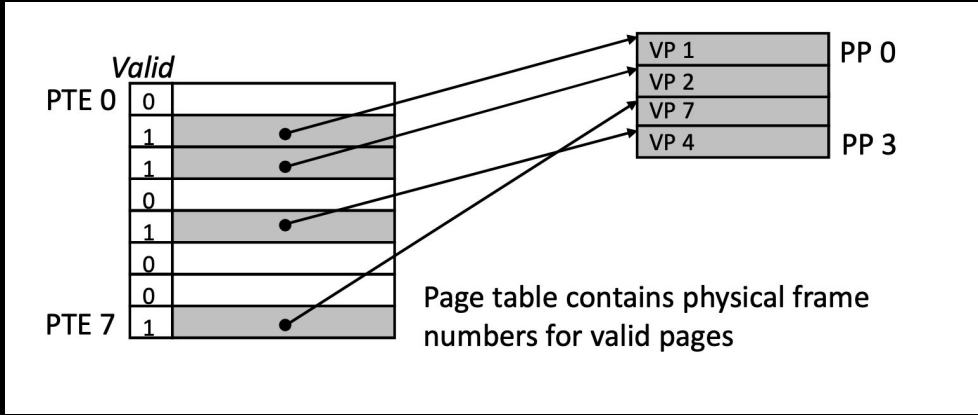

Page table
CPU
MMU
Virtual address space
Physical address space
Page
Fetch address 5KB
Fetch address Y+1KB
Address = X
Address = Y
Address = Z
Virtual address
Page number
Offset
Using page table
Physical address
Frame number
Offset

Page table structure

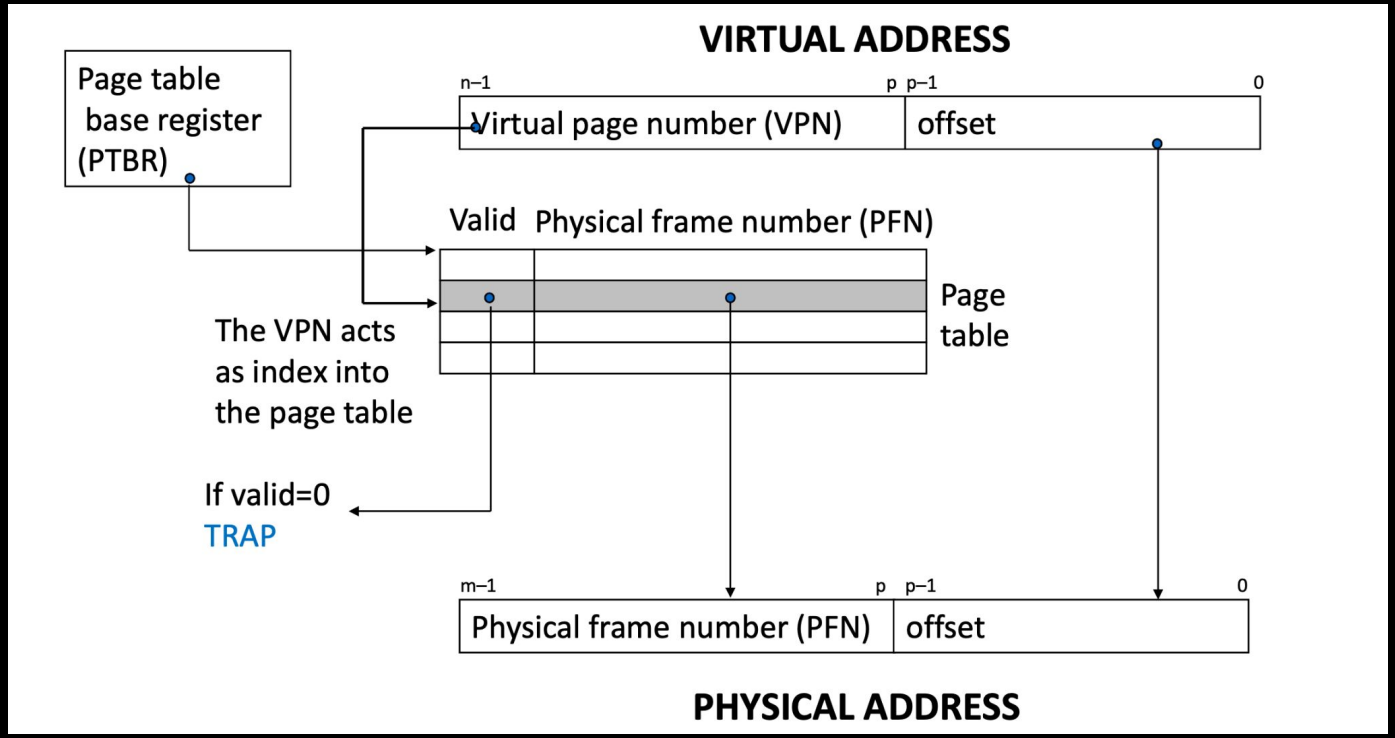
- Page table is an array, where i-th entry contains the information (physical frame number etc) of the i-th page of the process
- Page table has entries for all pages in address space, even those for which there is no physical frame number
    - *It is an array with fixed number of entries, not a dynamic data structure*
    - *Why this design? Why not a dynamic data structure whose size depends on number of used pages? (Hint: think of effort to traverse the data structure in MMU hardware)*
- Page table structure fixed for a given architecture, as the logic to traverse the page table is baked into MMU hardware

Page table entry

- Page table is array of page table entries, one per page of process
- i-th page table entry (PTE) contains physical frame number and other details (permissions, status, ..) of i-th page of process
  - *Valid: is this page in use by process (not all virtual addresses are used by process)*
  - *Various permission bits (more later)*
  - *Other status bits: present, dirty, accessed (more later)*



Address translation in MMU



Size of page tables

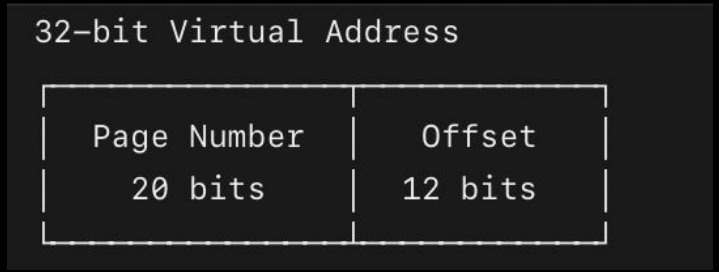
- What is typical size of page table in a 32-bit system?
- 2^32 = 4GB virtual address space
- Assume page size = 4KB = 2^12
- Number of PTEs = number of pages in virtual address space = (2^32/2^12) = 2^20 = 1M
- If each PTE is 4 bytes, page table size = 4 bytes * 1M entries = 4MB
- How are page tables stored in memory?
    - *All memory is only allocated in 4KB chunks, so how to store 4MB?*
- Solution: split page table into pages (much like memory image), use another page table to keep track of original page table!

Practice

1. A system has a 32-bit virtual address and a 4 KB page size. How many bits of the virtual address are used as the page offset?
A. 10 bits
B. 12 bits
C. 32 bits
D. 20 bits

Practice

1. A system has a 32-bit virtual address and a 4 KB page size. How many bits of the virtual address are used as the page offset?
A. 10 bits
B. 12 bits
C. 32 bits
D. 20 bits



Correct Answer: B. 12 bits

Explanation:

4 KB = 4096 bytes = 2^{12} bytes.

Therefore, 12 bits are required to identify a byte within a page.

Practice

2. With a 32-bit virtual address and 4 KB pages, how many virtual pages are in the complete virtual address space?

A. 2^{20} pages
B. 2^{32} pages
C. 2^{12} pages
D. 2^{40} pages

Practice

2. With a 32-bit virtual address and 4 KB pages, how many virtual pages are in the complete virtual address space?
A. 2^{20} pages
B. 2^{32} pages
C. 2^{12} pages
D. 2^{40} pages

Correct Answer: A. 2^{20} pages

Practice

3. For a 32-bit virtual address, 4 KB pages, and 4-byte PTEs, which statements are correct? Select every correct answer.

A. A single-level page table requires 4 MB.

B. The virtual address space is 4 GB.

C. Each PTE represents 4 KB of physical RAM.

D. There are 1,048,576 virtual pages.

Practice

3. For a 32-bit virtual address, 4 KB pages, and 4-byte PTEs, which statements are correct? Select every correct answer.
A. A single-level page table requires 4 MB.
B. The virtual address space is 4 GB.
C. Each PTE represents 4 KB of physical RAM.
D. There are 1,048,576 virtual pages.

Correct Answers: A, B and D
A. A single-level page table requires 4 MB. ✓
There are 2^{20} virtual pages and each PTE is 4 bytes.
- $2^{20} \times 4$ bytes = $2^{20} \times 2^2 = 2^{22}$ bytes = 4 MB

B. The virtual address space is 4 GB. ✓
- 2^{32} bytes = 4 GB

C. Each PTE represents 4 KB of physical RAM. ✗
A PTE maps a virtual page to a physical frame and contains additional status information. The 4 KB refers to the page/frame size, not the size of the PTE itself.
D. There are 1,048,576 virtual pages. ✓
- 2^{20} = 1,048,576

Practice

4. A virtual address is 5 KB and the page size is 4 KB. What are its page number and offset?
A. Page 1, offset 5 KB
B. Page 5, offset 1 KB
C. Page 0, offset 5 KB
D. Page 1, offset 1 KB

Practice

4. A virtual address is 5 KB and the page size is 4 KB. What are its page number and offset?
A. Page 1, offset 5 KB
B. Page 5, offset 1 KB
C. Page 0, offset 5 KB
D. Page 1, offset 1 KB

Correct Answer: D. Page 1, offset 1 KB

Virtual Address = 5 KB

Page 0 → 0 KB – 4 KB

Page 1 → 4 KB – 8 KB

Practice

5. For a 4 KB page, the offset identifies the position of the requested byte within that page.
True
False

Practice

5. For a 4 KB page, the offset identifies the position of the requested byte within that page.
True
False

True

Practice

6. A page table contains the mapping Page 1 → Frame 20. A CPU requests virtual address 5 KB on a system with 4 KB pages. What physical location is accessed?
A. Frame 20 + 5 KB offset
B. Frame 20 + 1 KB offset
C. Frame 5 + 1 KB offset
D. Frame 21 + 0 KB offset

Practice

6. A page table contains the mapping Page 1 → Frame 20. A CPU requests virtual address 5 KB on a system with 4 KB pages. What physical location is accessed?
A. Frame 20 + 5 KB offset
B. Frame 20 + 1 KB offset
C. Frame 5 + 1 KB offset
D. Frame 21 + 0 KB offset

Correct Answer: B. Frame 20 + 1 KB offset

Virtual: Page 1 + 1 KB

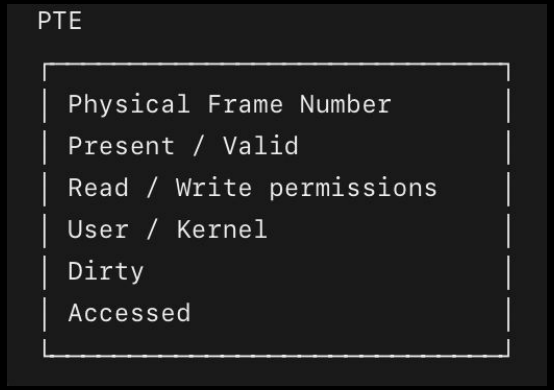
↓

Physical: Frame 20 + 1 KB

Practice

7. What is the best description of a Page Table Entry (PTE)?
A. The entire mapping table for a process
B. A hardware cache containing recent translations
C. A physical memory page containing application data
D. One entry containing the mapping and status information for a virtual page

Practice



7. What is the best description of a Page Table Entry (PTE)?
A. The entire mapping table for a process
B. A hardware cache containing recent translations
C. A physical memory page containing application data
D. One entry containing the mapping and status information for a virtual page

Answer

D. One entry containing the mapping and status information for a virtual page

Page Table = collection of PTEs

PTE = one mapping entry

Practice

8. A 4-byte PTE must be 4 bytes simply because the virtual address is 32 bits.
True
False

Practice

8. A 4-byte PTE must be 4 bytes simply because the virtual address is 32 bits.
True
False

False

Explanation:

PTE size is architecture-dependent.

A 32-bit virtual address does not automatically imply that the PTE must be 4 bytes.

Practice

9. With 4-byte PTEs and 4 KB pages, how many PTEs fit inside one 4 KB page of physical memory?
A. 512 PTEs
B. 4096 PTEs
C. 1024 PTEs
D. 256 PTEs

Practice

9. With 4-byte PTEs and 4 KB pages, how many PTEs fit inside one 4 KB page of physical memory?
A. 512 PTEs
B. 4096 PTEs
C. 1024 PTEs
D. 256 PTEs

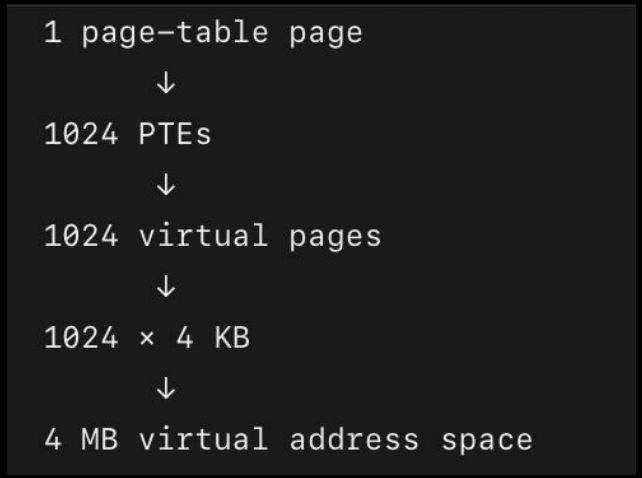
Correct Answer: C. 1024 PTEs

A physical page contains: 4 KB = 4096 bytes

Each PTE is: 4 bytes

Therefore: 4096 / 4 = 1024 PTEs

So one 4 KB page can store 1024 PTEs.



Practice

10. Why are multi-level page tables useful in a system with a 32-bit virtual address, 4 KB pages, and 4-byte PTEs?
A. They eliminate the need for physical frames.
B. They make each virtual page larger than 4 KB.
C. They avoid allocating page-table structures for unused portions of the virtual address space.
D. They guarantee that every memory access needs only one RAM access.

Practice

10. Why are multi-level page tables useful in a system with a 32-bit virtual address, 4 KB pages, and 4-byte PTEs?
A. They eliminate the need for physical frames.
B. They make each virtual page larger than 4 KB.
C. They avoid allocating page-table structures for unused portions of the virtual address space.
D. They guarantee that every memory access needs only one RAM access.

Correct Answer: C. They avoid allocating page-table structures for unused portions of the virtual address space.

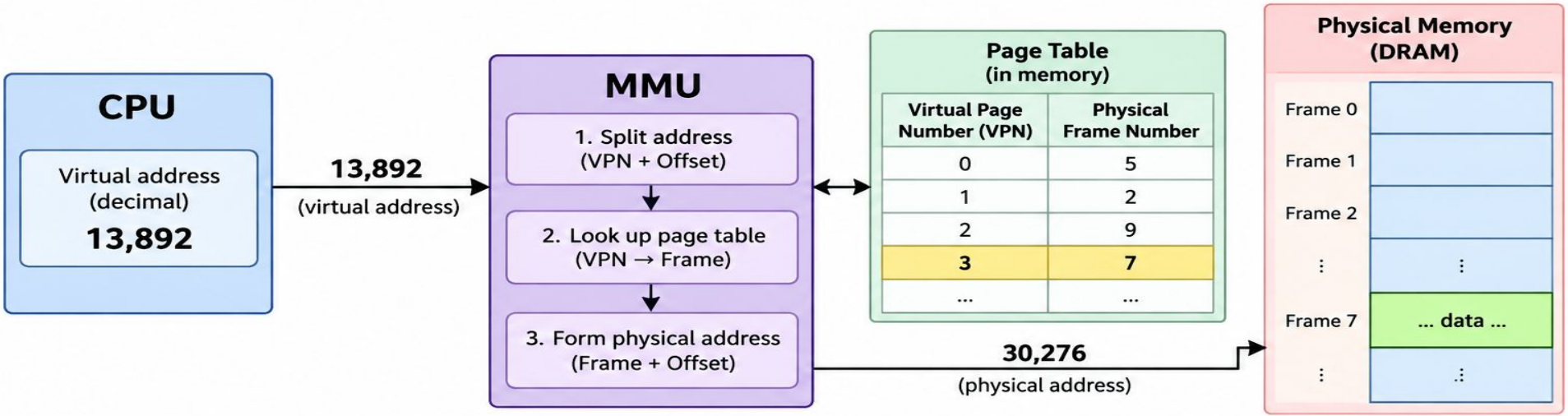
Explanation:

A 32-bit address space is: 4 GB

With 4 KB pages, there are: 1,048,576 virtual pages

A single-level page table therefore needs: 1,048,576 × 4 bytes = 4 MB

But most processes do not use the entire 4 GB virtual address space. Multi-level paging allows us to allocate lower-level page tables only for regions that are actually used.



1 Split the virtual address

Virtual address = 13,892
Page size = 4 KB = 4,096 bytes

VPN = 13,892 / 4,096 = 3
Offset = 13,892 % 4,096 = 1,604

Virtual address (13,892)

Virtual Page Number (VPN)	Offset
3	1,604

2 Look up the page table

VPN 3 maps to physical frame 7.

VPN	Frame
0	5
1	2
2	9
3	7
...	...

3 Form the physical address

Physical address =
Frame × Page size + Offset

= 7 × 4,096 + 1,604
= 28,672 + 1,604
= 30,276

Physical address (30,276)

Physical Frame Number	Offset (unchanged)
7	1,604

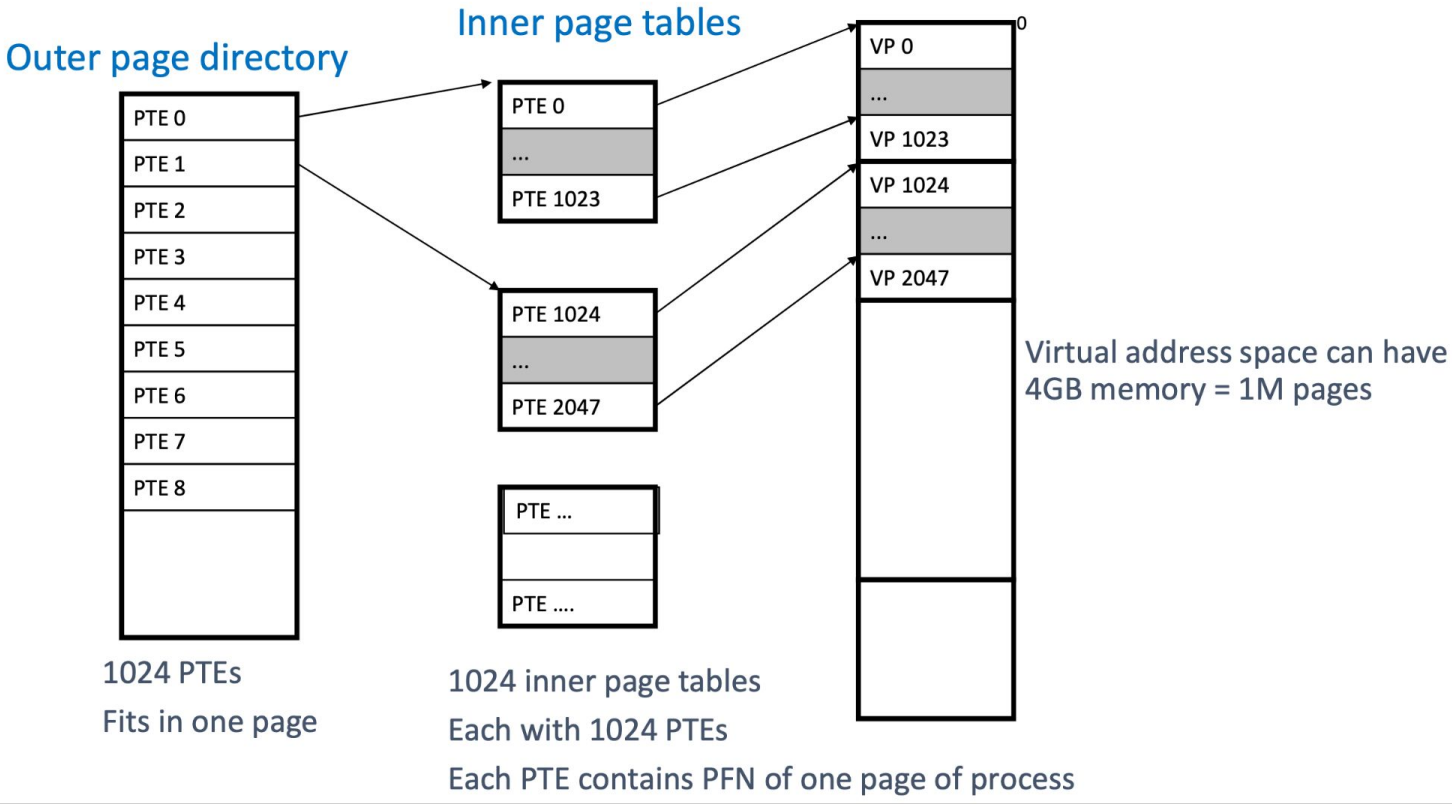
4 Access physical memory

The MMU sends physical address **30,276** to DRAM, which is in frame 7 at offset 1,604, and the data is read/written.

Key idea: The MMU replaces the virtual page number with a physical frame number, while keeping the offset unchanged.

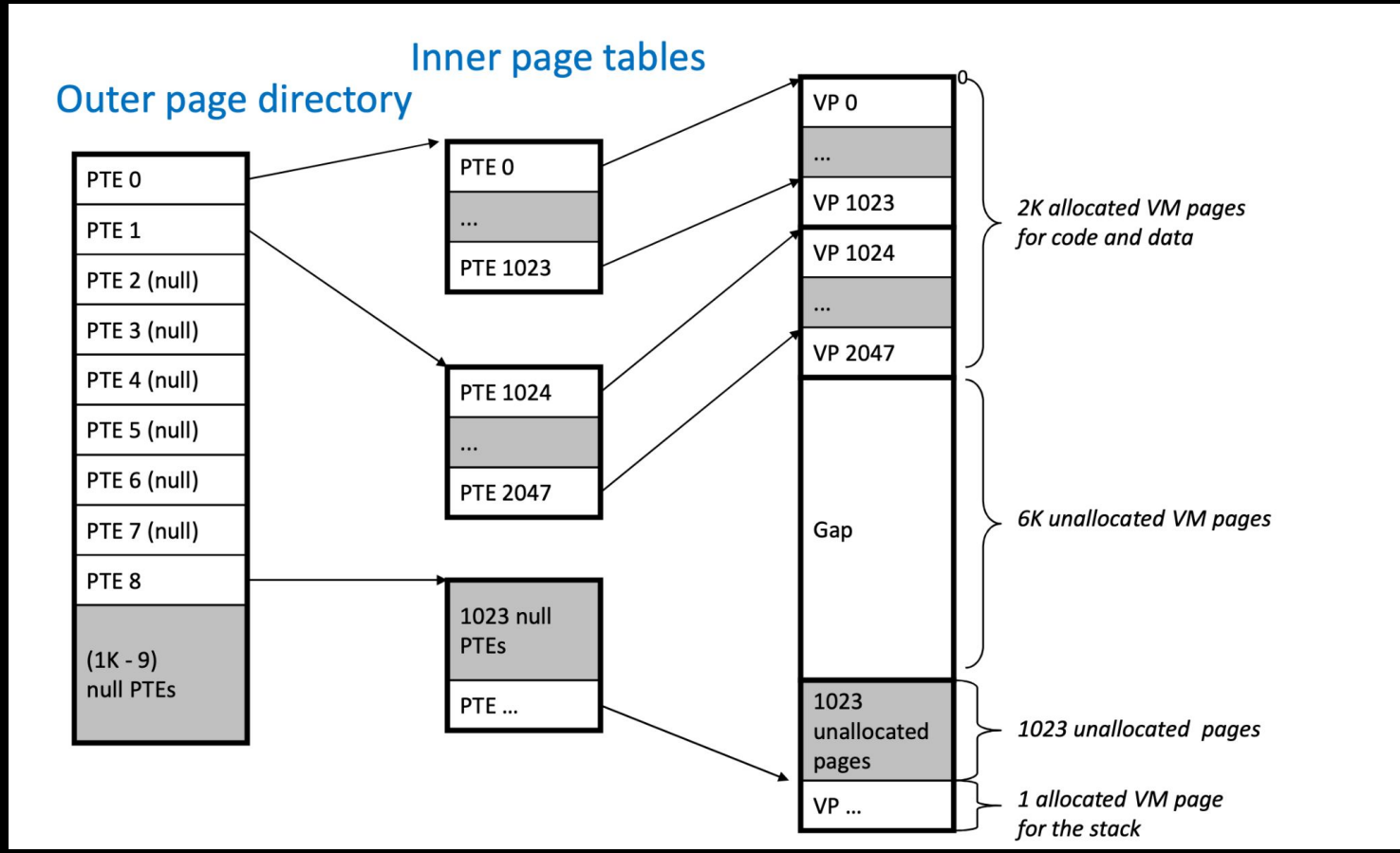
Two-level page table in 32-bit systems

- 4MB page table split into 1024 chunks of 4KB each (to fit in page)
- 1M PTEs split across 1024 pages, each containing 1024 PTEs
- Physical frame numbers of these 1024 chunks stored in an outer page table or page directory
  - *4 byte page table entry each, so outer page directory fits in one page here*
- Page table now has two levels
  - *Outer page table (page directory) has physical frame numbers of 1024 “inner” page table pages*
  - *Each inner page table has physical frame numbers (PTEs) of 1024 pages of the process virtual address space*


Outer page directory
Inner page tables
PTE 0
PTE 1
PTE 2
PTE 3
PTE 4
PTE 5
PTE 6
PTE 7
PTE 8
PTE 0
...
PTE 1023
PTE 1024
...
PTE 2047
PTE ...
PTE ....
0
VP 0
...
VP 1023
VP 1024
...
VP 2047
Virtual address space can have
4GB memory = 1M pages
1024 PTEs
Fits in one page
1024 inner page tables
Each with 1024 PTEs
Each PTE contains PFN of one page of process

Inner page tables on demand

- Note: not all inner page tables need to be created always, only those with at least one valid entry needed
- Example: Process with 2K pages of code+data, 6K + 1023 unallocated pages in address space, then one page allocated for stack
    - *First two inner page tables are allocated, hold the 2K valid PTEs*
    - *Next 6 inner page tables are not created, the corresponding entries in outer page directory are invalid / null*
    - *In next inner page table, 1023 invalid entries and one valid PTE containing frame number of stack page*
    - *Remaining inner page tables not created, corresponding outer page directory entries are invalid*

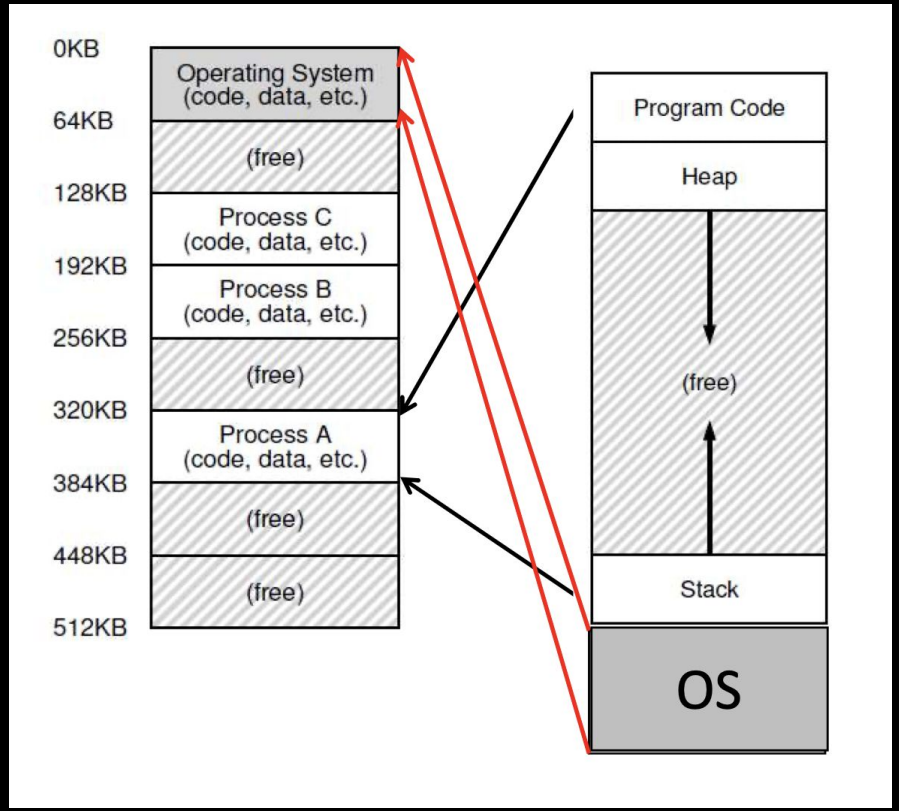

Inner page tables
Outer page directory
PTE 0
PTE 1
PTE 2 (null)
PTE 3 (null)
PTE 4 (null)
PTE 5 (null)
PTE 6 (null)
PTE 7 (null)
PTE 8
(1K - 9)
null PTEs
PTE 0
...
PTE 1023
PTE 1024
...
PTE 2047
1023 null
PTEs
PTE ...
0
VP 0
...
VP 1023
VP 1024
...
VP 2047
Gap
1023
unallocated
pages
VP ...
2K allocated VM pages
for code and data
6K unallocated VM pages
1023 unallocated pages
1 allocated VM page
for the stack

Revisit Virtual Address Space

- What should virtual address space/page table of process have? Any memory that the process needs to access during its execution
  - *Its own memory image: code, data, stack, heap*
  - *Other common memory it needs to access: shared language libraries, OS*
- Why? MMU allows access to memory only via virtual addresses
  - *Can only access physical memory mapped in page table at some virtual address*
  - *So all physical memory needed by process should be mapped into address space*
- OS binary image (kernel code, data) is mapped into the virtual address space of every process at addresses not used by process (high VA)
- Why is this done? Easy to jump to OS code during a trap

OS is part of address space of every process

- OS is not a separate process with its own address space
- Instead, OS code is part of the address space of every process
- A process sees OS as part of its code (e.g., like a library)
- During trap, process jumps to high virtual address



OS is part of address space of every process

- OS code/data assigned high virtual addresses
  - *Compiler ensures high virtual addresses not used by user code*
- OS virtual addresses are mapped to physical addresses of OS via page table entries of every process
- There is only one copy of OS code/data in RAM
  - *Loaded into RAM at low physical addresses during system bootup*
- Page tables of all processes map to same OS physical addresses
  - *Same high virtual addresses map to same physical addresses of OS code*

Page-level isolation and security

- How is OS code/data protected from illegal access by user?
- Page table has permissions for every memory page
  - *Whether read/write or read-only (code pages are read-only)*
  - *Whether page can be accessed in user mode or kernel mode*
- Page table mappings for OS code are protected to allow access only when CPU is in kernel mode
  - *CPU in user mode cannot access high virtual addresses of OS code*
  - *CPU in kernel mode (after trap instruction) can access OS code/data*
- MMU traps to OS if any violation detected during memory access, ensures user programs can only access memory they are permitted to access

File backed, anonymous, dirty pages

- Pages in the memory image of a process are of two types
  - *File-backed pages contain data from files on disk (e.g., page with executable code)*
  - *Anonymous pages are not backed by files on disk (e.g., pages containing stack, heap)*
- Further classification into dirty and non-dirty pages
- Dirty pages: pages whose content is different from their copy on disk
  - *E.g., file-backed pages whose contents have changed from file*
  - *E.g., anonymous pages whose contents have changed since last read from swap*
- PTE has information to track these types of pages, including dirty bit